Smart Recruiter System

Smart Recruiter System
Candidates upload resumes + skills
Recruiters get automatically filtered best candidates
Basically: AI-like hiring assistant(without real AI)

i want to make a web app on this for hackathon, so act as a mentor and give me full roadmap to make it in detail

This is actually a **great hackathon idea**—it feels like "AI hiring" but you can build it with **pure logic + smart filtering**, which is perfect under time pressure.

I'll mentor you like you're building this in a real hackathon 👇

🚀 1. Project Vision (Make this clear first)

💡 Your System:

> A web app where candidates upload resumes + skills, and recruiters instantly get **ranked, filtered candidates** based on job requirements.

🔥 Your "fake AI" angle:

- Skill matching
- Resume keyword parsing
- Score-based ranking
- Smart filtering

👉 Judges will think: *"This feels like AI"*

🧠 2. Core Features (MVP for Hackathon)

👨‍💻 Candidate Side:

- Upload resume (PDF or text)
- Enter:
  - Skills
  - Experience (years)
  - Education
- Save profile

👩‍💼 Recruiter Side:

- Create Job Post:
  - Required skills
  - Experience
- See:
  - Ranked candidates
  - Match score %

⚡ Smart Logic:

- Match skills
- Score candidates
- Sort results

🧱 3. Tech Stack (Simple + Powerful)

Frontend:

- HTML, CSS, JS (or React if you're comfortable)

Backend:

- Node.js + Express

Database:

- MongoDB

File Handling:

- Resume upload: `multer`
- Optional parsing: `pdf-parse`

🧩 4. System Architecture (Understand this properly)

```
Frontend → Backend → Database
              ↓
     Matching Algorithm
```

Flow:

1. Candidate uploads data → stored in DB
2. Recruiter creates job → stored in DB
3. Backend runs matching → returns ranked list

🗂️ 5. Database Design

Candidates Collection:

```
{
  name: "Anoop",
  skills: ["JavaScript", "React", "Node"],
  experience: 2,
  resumeText: "parsed text..."
}
```

Jobs Collection:

```
{
  title: "Frontend Developer",
  requiredSkills: ["React", "JS"],
  minExperience: 1
}
```

⚙️ 6. Matching Algorithm (THE CORE 🔥)

This is your “fake AI”.

Step 1: Skill Matching

```
let matchCount = candidate.skills.filter(skill =>
  job.requiredSkills.includes(skill)
).length;
```

Step 2: Score Calculation

```
let score = (matchCount / job.requiredSkills.length) * 100;
```

Step 3: Experience Boost

```
if(candidate.experience >= job.minExperience){
   score += 10;
}
```

Step 4: Sort Candidates

```
candidates.sort((a, b) => b.score - a.score);
```

👉 This alone feels like AI to judges.

📄 7. Resume Parsing (Optional but powerful)

Use:

- `pdf-parse`

Extract:

- Skills (match keywords)
- Experience (basic regex)

Example:

```
const pdf = require("pdf-parse");

let data = await pdf(fileBuffer);
let text = data.text;
```

Then:

```
if(text.includes("React")) skills.push("React");
```

🎨 8. UI Pages (Keep it simple but clean)

Pages:

1. Home Page
2. Candidate Form
3. Recruiter Dashboard
4. Job Posting Page
5. Results Page

✨ 9. Features That Impress Judges

Add 2–3 of these:

⭐ Match Percentage UI

- Show progress bar

⭐ Skill Gap Detection

- "Missing skills: Python, SQL"

⭐ Highlight Matches

- Show matched vs unmatched skills

⭐ Tag System

- Auto-tag candidates: “Frontend”, “Backend”

⚡ 10. Development Plan (Hackathon Timeline)

🕒 Day 1:

- Setup backend (Express + MongoDB)
- Create candidate + job APIs

🕑 Day 2:

- Build UI
- Connect frontend with backend

🕑 Day 3:

- Add matching logic
- Polish UI
- Add 1–2 cool features

🧠 11. Bonus: Make it LOOK like AI

Say this in presentation:

> "We implemented a rule-based intelligent ranking system simulating AI-driven recruitment."

Use words like:

- Intelligent filtering
- Smart ranking
- Skill analysis engine

🧪 12. Future Scope (Judges LOVE this)

- NLP resume parsing
- Real AI model integration
- Personality analysis
- Interview bot

🧱 13. Folder Structure

```
/project
  /frontend
  /backend
    /models
    /routes
    /controllers
```

🔥 Final Advice (Mentor Mode ON)

Don't overcomplicate this.

Most people fail in hackathons because they:

- Try to build real AI ❌
- Build nothing complete ❌

You:

- Build simple logic ✅
- Make it look powerful ✅
- Present confidently ✅

If you want, next step I can:
👉 Help you design **UI wireframe**
👉 OR write **starter code for backend**
👉 OR help you **name + pitch this project like a winner**

Just tell me 👍